Waste classifier

Klasifikátor odpadu pomocí konvoluční neuronové sítě

Jakub Šrámek

C4a

Obsah

Obsah 2
1. Úvod a cíl projektu 3
1.1 Klasifikační kategorie 3
2. Data a jejich původ 3
2.1 Sběr dat - video_to_frames.py 3
2.2 Augmentace dat - augment.py 3
2.3 Extrakce příznaků - build_dataset.py 3
3. Model strojového učení 5
3.1 Architektura CNN — train.py 5
3.2 Trénovací konfigurace 5
3.3 Nasazení - convert_to_tflite.py 5
4. Architektura projektu 6
4.1 Softwarová architektura — Pipeline pattern 6
4.2 Webová aplikace - App.py 6
Klíčové komponenty App.py 6
Kompatibilita TFLite knihovny 6
4.3 Datový tok při inferenci 6
5. Spustičování aplikace 7
5.1 Požadavky 7
5.2 Instalace a spuštění 7
5.3 Postup trénování od začátku 7
6. Popis modulů 8
video_to_frames.py 8
augment.py 8
build_dataset.py 8
train.py 8
App.py 8
7. Hodnocení modelu 9
7.1 Metriky 9
7.2 Monitorování trénování 9
8. Zhodnocení projektu 9
8.1 Co se podařilo 9
8.2 Možné vylepšení 9

1. Úvod a cíl projektu

Waste_classifier je softwarový projekt, který využívá strojové učení k automatické klasifikaci odpadu do pěti kategorií. Cílem je pomoci uživatelům správně třídit odpad pomocí fotografie pořízené mobilním telefonem nebo webkamerou.

Projekt byl vytvořen v rámci předmětu PV.

1.1 Klasifikační kategorie

Kategorie	Popelnice	Barva	Příklady
Plasty	Žlutá popelnice	Žlutá	PET lahve, sáčky, plastové obaly
Papír	Modrá popelnice	Modrá	Kartony, noviny, lepenka
Sklo	Zelená popelnice	Zelená	Lahve, sklenice, sklenice od džemu
Bio	Hnědá popelnice	Hnědá	Slupky ovoce, zbytky jídla, čajové sáčky
Směsný	Černá popelnice	Černá	Znečištěné obaly, nevyhovující odpad

2. Data a jejich původ

Tento projekt využívá výhradně vlastnoručně pořízená data.Snímky byly nahrány pomocí mobilního telefonu jako krátká videa nebo fotografie různých druhů odpadu v různých prostředích a světelných podmínkách.

2.1 Sběr dat - video_to_frames.py

Z každého videa byl každý 8. snímek extrahován jako JPEG obrázek o rozlišení 512×512 px. Videa jsou rozdělena do složek podle kategorie odpadu.

- Formát vstupních videí: MP4, MOV, AVI, MKV
- Interval extrakce: každý 8. snímek (parametr FRAME_INTERVAL = 8)
- Výstupní rozlišení: 512×512 px (zachován poměr stran)

2.2 Augmentace dat - augment.py

Každý originální snímek je zpracován augmentační pipeline, která z něj generuje až 17 variant. Účelem je zvýšit robustnost modelu a snížit přeučení.

Aplikované augmentace:

- Otáčení: 90°, 180°, 270° a náhodný úhel
- Horizontalní zrcadlení
- Změna jasu a kontrastu (úroveň 0,6–1,4)
- Gaussovo rozmazatí (poloměr 0,5 a 1,0)
- Náhodné řezy (crop) z okrajů
- Gaussovský šum (sigma 5–12)
- Barevný jitter (posun RGB kanálů)
- Náhradní pozadí (pokud jsou k dispozici textury)

2.3 Extrakce příznaků - build_dataset.py

Pro každý obrázek je vypočteno 15 numerických příznaků popisujících barvu, texturu a hrany. Tyto příznaky jsou uloženy do CSV souboru a zobrazovány v UI jako „Image Properties“.

Příznak	Popis	Technický výpočet
brightness	Střední jas pixelu	0.299R + 0.587G + 0.114B
contrast	Kontrast (rozptyl jasu)	std(brightness)
saturation	Střední sytost	(max–min) / max (HSV)
color_uniformity	Rovnoměrnost barev	std(saturation)
warm_ratio	Podíl teplých barev	píxely kde R > B + 15
transparency	Průhlednost/bělost	podíl pixelu s jasem > 210
dark_ratio	Podíl tmavých pixelu	podíl pixelu s jasem < 40
edge_density	Hustota hran	mean(FIND_EDGES filter)
edge_intensity	Intenzita hran	std(FIND_EDGES filter)
texture_roughness	Drsnost textury	std(DETAIL filter)
smoothness	Hladkost povrchu	std(EMBOSS filter)
entropy	Entropie jasu	$-\Sigma\ p \cdot \log_2(p)$, histogram 64 binů
edge_entropy	Entropie hran	$-\Sigma\ p \cdot \log_2(p)$, histogram 32 binů
channel_variance	Variance RGB kanálů	var(mean_R, mean_G, mean_B)
highlights	Světelné odlesky	podíl pixelu s jasem > 240

3. Model strojového učení

3.1 Architektura CNN — train.py

Model je konvoluční neuronová síť (CNN) implementovaná v TensorFlow/Keras. Vstupem jsou barevné obrázky 128×128 px, výstupem jsou pravděpodobnosti pěti tříd.

Vrstva	Parametry	Účel
Conv2D	32 filtrů, jádro 3×3	Extrakce nízkoúrovněových příznaků (hrany, textury)
BatchNorm + ReLU	—	Normaliz. aktivací, zabránění explodujícímu gradientu
MaxPooling2D	pool 2×2	Redukce prostorové dimenze 2×
Conv2D	64 filtrů, jádro 3×3	Extrakce střednířúrovněových příznaků
BatchNorm + ReLU + Pool	—	Normaliz. + redukce
Conv2D	128 filtrů, jádro 3×3	Extrakce vysokoúrovněových příznaků
BatchNorm + ReLU + Pool	—	Normaliz. + redukce
Flatten	—	Převod 3D tenzoru na 1D vektor
Dense	256 neuronů + ReLU	Plně propojená vrstva
Dropout	p = 0,5	Regularizace, předejde přeučení
Dense + Softmax	5 neuronů	Výstupní pravděpodobnosti pro 5 tříd

3.2 Trénovací konfigurace

Trénování probíhá s následujícím nastavením:

- Optimalizátor: Adam (adaptivní míra učení)
- Ztrátová funkce: categorical_crossentropy
- Velikost dávky (batch size): 32
- Maximální počet epoch: 5 (s EarlyStopping patience = 5)
- Val/train split: 80 % trénovací, 20 % testovací data
- Váhy tříd: vyvážení pomocí compute_class_weight (sklearn)
- Vyvážení datasetu: každá kategorie obsahuje stejný počet snímků (min_count)

3.3 Nasazení - convert_to_tflite.py

Po trénování je model konvertován do formátu TensorFlow Lite (.tflite) pro optimalizovanou inferenci na běžných PC bez GPU. TFLite model je přibližně 4× menší a rychlejší než originál.

- Vstup: model_cnn.keras (Keras formát)
- Výstup: model_cnn.tflite
- Konvertor: tf.lite.TFLiteConverter.from_keras_model

4. Architektura projektu

4.1 Softwarová architektura — Pipeline pattern

Projekt využívá architektonických vzor Pipeline (rourová linka). Jednotlivé moduly jsou nezávislé a každý je zodpovědný za jednu fázi zpracování dat:

Modul (soubor)	Fáze pipeline	Vstup → Výstup
video_to_frames.py	1. Sběr dat	Videa (MP4) -> JPEG snímky
augment.py	2. Augmentace	Originální JPEG -> 17× více JPEG
build_dataset.py	3. Extrakce příznaků	JPEG snímky -> dataset.csv
train.py	4. Trénování CNN	JPEG + CSV -> model_cnn.keras
convert_to_tflite.py	5. Export modelu	model_cnn.keras -> model_cnn.tflite
App.py	6. Nasazení (produkce)	HTTP + fotografie -> klasifikace JSON

4.2 Webová aplikace - App.py

Aplikace je webový server postavený na frameworku Flask. Webové rozhraní je navrženo jako single-page application (SPA) bez externích front-end frameworků.

Klíčové komponenty App.py

- TFLite Interpreter - načte model při startu aplikace, alokuje tenzory
- extract_features_for_display() - vypočte 15 příznaků pro zobrazení v UI
- / route - vyrenderuje HTML stránku se všemi metadaty
- /classify route - přijme POST s fotografií, provede inferenci, vrátí JSON

Kompatibilita TFLite knihovny

App.py podporuje tři varianty importu TFLite interpreteru v pořadí priorit:

- ai_edge_litert (moderní nástupce tflite_runtime)
- tflite_runtime (lehká varianta bez plného TF)
- tensorflow.lite (plný TensorFlow jako záloha)

4.3 Datový tok při inferenci

Průběh klasifikace po nahrání fotografie uživatelem:

- Uživatel vybere/nasnímá fotku v prohlížeči
- JS odešle POST /classify (multipart/form-data)
- Flask přijme soubor, otevře jako PIL Image (RGB)
- extract_features_for_display() — výpočet 15 příznaků pro UI
- Obrázek je změněn na 128×128 px, normalizován (/ 255), rozšířen o batch dimenzi
- TFLite interpreter spustí inferenci
- Vrní JSON: kategorie, confidence %, pravděpodobnosti všech tříd, příznaky
- JS zobrazí výsledek s barevnými pruhy a tipy na třídění

5. Spustičování aplikace

5.1 Požadavky

Požadavek	Detail
Python	≥ 3.9
Knihovny	flask, numpy, pillow, ai_edge_litert nebo tflite_runtime nebo tensorflow
Trénování (volitelně)	tensorflow, scikit-learn, pandas, opencv-python
Paměť	Min. 512 MB RAM pro inferenci
Platforma	Windows (otestováno na školních PC)

5.2 Instalace a spuštění

Postup pro spuštění na školním PC bez IDE:

- 1. Nainstalovat závislósti:
  ```
  pip install flask numpy pillow ai_edge_litert
  ```
- 2. Přejdi do složky se zdrojovými soubory:
  ```
  cd src
  ```
- 3. Spustit aplikaci:
  ```
  python App.py
  ```
- 4. Otevřít prohlížeč na adrese:
  ```
  http://localhost:5000
  ```

5.3 Postup trénování od začátku

Pro opakované trénování modelu je potřeba spustit scripty v pořadí:

```
python video_to_frames.py    # 1. Extrakce snímků z videí
python augment.py            # 2. Generování augmentovaných snímků
python build_dataset.py      # 3. Vytvoření CSV datasetu
python train.py              # 4. Trénování CNN modelu
python convert_to_tflite.py  # 5. Export do TFLite
python App.py               # 6. Spuštění webové aplikace
```

6. Popis modulů

video_to_frames.py

Funkce	Popis
extract_frames(video_path, output_dir)	Extrahuje snímky z videa. Vrací počet uložených souborů.
main()	Prochází složky kategorií a volá extract_frames pro každé video.

augment.py

Funkce	Popis
center_crop(img, crop_pct)	Orizne obrázek ze středu o dané procento.
random_crop(img)	Náhodný řez pro simulaci různých vzdadáleností kamery.
add_noise(img)	Přidá Gaussovský šum.
color_jitter(img)	Náhodný posun RGB kanálů.
random_background(img)	Nahradí pozadí texturou ze složky backgrounds/.
augment_image(img)	Hlavní pipeline — vrátí seznam všech augmentovanch variant.
main()	Projde data/own/, augmentuje, uloží do data/augmented/.

build_dataset.py

Funkce	Popis
extract_features(img)	Vypočte 15 numerických příznaků z obrázku. Vrací dict.
main()	Prochází augmented/ a vytvoří dataset.csv.

train.py

Skript (bez třídové struktury) trénuje CNN. Spustí se jako modul, projít všechny kroky a uloží model + metadata.

App.py

Funkce / route	Popis
extract_features_for_display(img)	Stejná logika jako build_dataset, přizpůsobená pro jedno zobrazeni v UI.
GET /	Renderuje HTML předlohu se všemi metadaty ze JSON souboru.
POST /classify	Inferenní endpoint. Vrací JSON s kategorií, confidence a příznaky.

7. Hodnocení modelu

7.1 Metriky

Model je hodnocen na testovací množině (20 % dat, stratifikovaný split). Použité metriky:

- Celková přesnost (accuracy) — zapísána do model_meta.json a zobrazena v UI
- Classification report (precision, recall, F1-score) — na konzoli při trénování
- Matice záměn (confusion matrix) — na konzoli při trénování

7.2 Monitorování trénování

Trénování využívá callback EarlyStopping který sleduje val_accuracy a zastaví trénování při absenci zlepšení po 5 epochách. Nejlepší váhy jsou automaticky obnoveny (restore_best_weights=True).

8. Zhodnocení projektu

8.1 Co se podařilo

- Celá pipeline od sběru dat až po webovou aplikaci je plně funkční
- Data jsou 100 % vlastní - pořízena videa různých druhů odpadu
- Augmentací bylo dosaženo násobného zvětšení datasetu
- CNN model dosáhuje relevantní přesnosti na testovacích datech
- Uživatelské rozhraní je přívětivé a intuitivní

8.2 Možné vylepšení

- Zvětšení datasetu dalšími nahrávkami v různých prostředích
- Přídaní tlačítka pro přímou kameru (getUserMedia API)
- Přidání všech zbylých konteinerů a typů odpadků